

Competitive Programming

Section 73: Convex Geometry And Minkowski Sums

Competitive Programming Course

Table of contents

1	The convex hull as a compression	3
2	Rotating calipers	4
3	Minkowski sums	5
4	Exactness and overflow	6
5	Worked trace: the hull stack removes information safely	7
6	Worked trace: why a calipers pointer never retreats	8
7	Worked trace: adding polygon boundaries	9
8	Recognition table	10
9	C++ and Python implementation notes	11
10	Proof habits and degeneracies	12
11	Exercises	13

1 The convex hull as a compression

For any objective maximized by a linear direction, interior points never win. The convex hull replaces an arbitrary point set by its extreme boundary. Andrew's monotone chain sorts points and maintains the invariant that every consecutive triple on a partial chain is a strict counter-clockwise turn. Popping while the cross product is nonpositive removes both right turns and redundant collinear boundary points.

2 Rotating calipers

For a fixed hull edge, the signed distance of hull vertices from its line is unimodal around a convex polygon. Its farthest supporting vertex therefore moves only forward as the edge moves forward. This monotone pointer is the rotating-calipers principle. For diameter, compare successive cross products against the current edge and advance while area increases. Every antipodal supporting pair is visited, so the maximum squared distance is found in linear time after the hull.

3 Minkowski sums

The Minkowski sum is $A+B = \{a+b\}$. A linear direction is maximized by adding the maximizers from A and B, so the result of two convex polygons is convex. After rotating both polygons to their lowest-leftmost vertex, their directed edge vectors are already sorted by polar angle. Merge those two cyclic lists; equal-angle vectors are added together. This is the same reason merging two sorted arrays works.

Membership in a convex polygon is logarithmic. Fix vertex zero, reject points outside its first/last rays, binary-search the fan triangle, then use one exact cross-product test. Treat boundary rays and degenerate hulls separately.

4 Exactness and overflow

Coordinates near $1e9$ make a cross product approach $4e18$. Check the full expression range, not merely one multiplication. Python integers are exact; C++ may require `__int128` if the stated bounds exceed signed 64-bit.

5 Worked trace: the hull stack removes information safely

Process sorted points $(0,0)$, $(1,0)$, $(1,1)$, $(2,0)$. After the first three, the lower stack contains a left turn. When $(2,0)$ arrives, the last three points turn clockwise, so $(1,1)$ is popped. This is not a heuristic: the popped point lies above the segment joining the surrounding boundary points and cannot be part of the lower supporting chain.

For collinear points $(0,0)$, $(1,0)$, $(2,0)$, popping on a nonpositive cross product keeps only the endpoints. Popping only on a negative cross product keeps boundary collinear points. Both policies can be correct, but every later algorithm must know which canonical hull it receives.

6 Worked trace: why a calipers pointer never retreats

Fix a directed hull edge. As a second pointer walks around a convex polygon, the triangle area with that edge increases until the opposite support line is reached, then decreases. When the fixed edge advances, the supporting direction rotates forward, so an optimal opposite vertex cannot move backward around the cyclic order. This is the real source of linear time: not “two pointers” by itself, but monotonicity of the maximizing support point.

Always decide how ties are handled. On parallel opposite edges, two adjacent vertices can have equal area; diameter code may need to test both candidate pairs even though the pointer advances through only one representation.

7 Worked trace: adding polygon boundaries

Suppose polygon A has successive edge directions east, north-west, south-west, and polygon B has east, north, west, south. Both edge lists are cyclically angle-sorted after rotating to their lowest-leftmost vertices. Merge them like sorted arrays. When both next edges point east, add the vectors and advance both lists; otherwise advance the edge with smaller polar angle.

The starting vertex of the result is the sum of the two starting vertices. After every merged vector is applied, the walk returns to its start because both original edge-vector sums are zero.

8 Recognition table

Objective or query	Convex reduction
maximum dot product or extreme direction	keep only the hull
farthest pair / width / support pair	rotating calipers
choose $\mathbf{a}+\mathbf{b}$ or $\mathbf{a}-\mathbf{b}$ from two convex sets	Minkowski sum or difference
many membership queries in one convex polygon	fan binary search

Minkowski difference is just $\mathbf{A} + (-\mathbf{B})$. In particular, polygons $\mathbf{A}$ and $\mathbf{B}$ intersect exactly when the origin belongs to $\mathbf{A}-\mathbf{B}$. Many collision and distance problems become questions about the origin relative to one convex polygon.

9 C++ and Python implementation notes

Use one `Point` type and one orientation convention throughout. In C++, cast before multiplication; casting a cross product after two overflowing `long long` products is too late. Python avoids overflow but can still lose exactness if integer predicates are converted to floats prematurely.

For cyclic algorithms, prefer modulo only at boundary access and keep pointer increments explicit. Normalize polygons to one documented starting vertex before merging. Remove the duplicated final vertex if a boundary walk returns to its start.

10 Proof habits and degeneracies

- A hull proof must show every popped point is unnecessary and every retained extreme point survives.
- A calipers proof must identify a unimodal quantity and explain why its maximizer moves monotonically as the first feature advances.
- A Minkowski proof is cleanest through support functions: $\text{support}(A+B, d) = \text{support}(A, d) + \text{support}(B, d)$
- A point-location proof must state whether boundary points count as inside.

Test one point, two points, all-collinear inputs where allowed, duplicate raw points, parallel polygon edges, touching boundaries, the origin on a Minkowski edge, and coordinates near the overflow limit.

11 Exercises

A establishes a canonical strict hull. B adds rotating calipers. C composes edge-angle merging with logarithmic point location. D is blind and asks you to turn a distance between two convex sets into a distance from the origin.